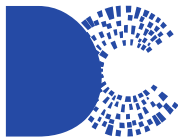




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Otimização de código

Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

Introdução

- ▶ Na ciência da computação, a eficiência de um sistema pode ser definida pelos recursos e tempo consumidos na realização de uma tarefa

$\uparrow \textit{Eficiência} \longleftrightarrow \textit{Recursos} \downarrow$

$\uparrow \textit{Eficiência} \longleftrightarrow \textit{Tempo} \downarrow$

Introdução

- ▶ Qual a história e por que era importante?
 - ▶ Os recursos computacionais eram muito limitados
 - ▶ Grande consumo de potência e uso compartilhado



Introdução

- ▶ Por que hoje é importante?
 - ▶ Restrições de custo
 - ▶ Baixo consumo de potência



Introdução

- ▶ Quais são as métricas para medir a eficiência computacional?
 - ▶ Espaço: alocação em memória
 - ▶ Tempo: número de passos executados

Introdução

- ▶ Como as otimizações de código podem ser feitas?
 - ▶ Diretivas de compilação

Introdução

- ▶ Como as otimizações de código podem ser feitas?
 - ▶ Diretivas de compilação
 - ▶ Eficiência algorítmica

Introdução

- ▶ Como as otimizações de código podem ser feitas?
 - ▶ Diretivas de compilação
 - ▶ Eficiência algorítmica
 - ▶ Análise de desempenho

Diretivas de compilação

- ▶ As ferramentas de desenvolvimento, como o GCC, permitem que o desenvolvedor controle que aspectos da aplicação deseja otimizar através da utilização de diretivas
 - ▶ Compromisso entre desempenho, tamanho de código e capacidade de depuração
 - ▶ Por padrão, nenhuma otimização é aplicada (-O0)

Diretivas de compilação

- ▶ As ferramentas de desenvolvimento, como o GCC, permitem que o desenvolvedor controle que aspectos da aplicação deseja otimizar através da utilização de diretivas
 - ▶ Compromisso entre desempenho, tamanho de código e capacidade de depuração
 - ▶ Por padrão, nenhuma otimização é aplicada (-O0)

↑ *Otimização* $\longleftrightarrow$ *Tempo de compilação* ↑

Diretivas de compilação

- ▶ Para ilustrar o impacto das diretivas de otimização na eficiência do sistema, considere a seguinte solução de ordenação por força bruta

```
1  #include <stdio.h>
2  #include <stdint.h>
3  #include <stdlib.h>
4  void trocar(int32_t* i, int32_t* j) {
5      int32_t k = *i;
6      *i = *j; *j = k;
7  }
8  void ordenar(int32_t* V, uint32_t n) {
9      for(uint32_t i = 0; i < n - 1; i++) {
10         uint32_t min = i;
11         for(uint32_t j = i + 1; j < n; j++)
12             if(V[j] < V[min]) min = j;
13         if(i != min) trocar(&V[i], &V[min]);
14     }
15 }
... ..
```

Diretivas de compilação

- Para ilustrar o impacto das diretivas de otimização na eficiência do sistema, considere a seguinte solução de ordenação por força bruta

```
1  #include <stdio.h>
...
16  int main() {
17      const uint32_t n = 100000;
18      int32_t* V = (int32_t*)(malloc(n *
19          sizeof(int32_t)));
20      for(uint32_t i = 0; i < n; i++)
21          V[i] = rand() * rand();
22      ordenar(V, n);
23      printf("min_=%i, max_=%i\n", V[0], V[n - 1]);
24      return 0;
}
```

```
$ gcc -Wall exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
```

Diretivas de compilação

- ▶ Para ilustrar o impacto das diretivas de otimização na eficiência do sistema, considere a seguinte solução de ordenação por força bruta

```
1  #include <stdio.h>
...
16 int main() {
17     const uint32_t n = 100000;
18     int32_t* V = (int32_t*)(malloc(n *
19         sizeof(int32_t)));
20     for(uint32_t i = 0; i < n; i++)
21         V[i] = rand() * rand();
22     ordenar(V, n);
23     printf("min_=%i, max_=%i\n", V[0], V[n - 1]);
24     return 0;
}
```

```
$ gcc -Wall exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 9.300 s +- 0.533 s [User: 9.230 s, System: 0.030 s]
Range (min ... max): 8.922 s ... 10.621 s 10 runs
```

Diretivas de compilação

- ▶ Otimização de nível 1 (-O1)
 - ▶ O compilador tenta reduzir tanto o tamanho do código gerado como o tempo de execução, sem impactar muito no tempo de compilação
 - ▶ -fdelayed-branch: explora as instruções do *delay slot*
 - ▶ -fguess-branch-probability: aplica heurísticas para determinar as probabilidades de desvios
 - ▶ -freorder-blocks: reorganiza blocos básicos para diminuir desvios e melhorar localidade

```
$ gcc -Wall -O1 exemplo.c -o exemplo.elf  
$ hyperfine --warmup 3 './exemplo.elf'
```

Diretivas de compilação

- ▶ Otimização de nível 1 (-O1)
 - ▶ O compilador tenta reduzir tanto o tamanho do código gerado como o tempo de execução, sem impactar muito no tempo de compilação
 - ▶ -fdelayed-branch: explora as instruções do *delay slot*
 - ▶ -fguess-branch-probability: aplica heurísticas para determinar as probabilidades de desvios
 - ▶ -freorder-blocks: reorganiza blocos básicos para diminuir desvios e melhorar localidade

```
$ gcc -Wall -O1 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
  Time (mean +- d): 8.253 s +- 0.078 s [User: 8.246 s, System: 0.001 s]
  Range (min ... max): 8.186 s ... 8.398 s 10 runs
```

Diretivas de compilação

- ▶ Otimização de nível 2 (-O2)
 - ▶ Amplia o esforço de -O1, sem realizar otimizações que envolvem o compromisso entre espaço e velocidade, aumentando tanto o tempo de compilação como o desempenho do código gerado
 - ▶ -finline-small-functions: através de heurísticas, o compilador decide sobre a integração da função no corpo da função que fez a chamada
 - ▶ -fstore-merging: combina escrita de valores imediatos em endereços consecutivos de memória, reduzindo o número de instruções
 - ▶ -fschedule-insns: reordena instruções para evitar paralisação de execução por espera de dados, como em operações de ponto flutuante

```
$ gcc -Wall -O2 exemplo.c -o exemplo.elf  
$ hyperfine --warmup 3 './exemplo.elf'
```


Diretivas de compilação

- ▶ Otimização de nível 2 (-O2)
 - ▶ Amplia o esforço de -O1, sem realizar otimizações que envolvem o compromisso entre espaço e velocidade, aumentando tanto o tempo de compilação como o desempenho do código gerado
 - ▶ -finline-small-functions: através de heurísticas, o compilador decide sobre a integração da função no corpo da função que fez a chamada
 - ▶ -fstore-merging: combina escrita de valores imediatos em endereços consecutivos de memória, reduzindo o número de instruções
 - ▶ -fschedule-insns: reordena instruções para evitar paralisação de execução por espera de dados, como em operações de ponto flutuante

```
$ gcc -Wall -O2 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 8.179 s +- 0.023 s [User: 8.175 s, System: 0.001 s]
Range (min ... max): 8.169 s ... 8.238 s 10 runs
```

Diretivas de compilação

- ▶ Otimização de nível 3 (-O3)
 - ▶ Realiza todas as otimizações de -O2 e aplica otimizações adicionais para desempenho
 - ▶ -finline-functions: todas as funções são consideradas para serem *inline*, ou seja, com integração do código no corpo da função que fez a chamada
 - ▶ -floop-unroll-and-jam: desenrola e combina laços para evitar desvios
 - ▶ -fsplit-paths: divide caminhos que levam a laços de retorno, melhorando a remoção de código morto e eliminação de subexpressão comum

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf  
$ hyperfine --warmup 3 './exemplo.elf'
```

Diretivas de compilação

- ▶ Otimização de nível 3 (-O3)
 - ▶ Realiza todas as otimizações de -O2 e aplica otimizações adicionais para desempenho
 - ▶ -finline-functions: todas as funções são consideradas para serem *inline*, ou seja, com integração do código no corpo da função que fez a chamada
 - ▶ -floop-unroll-and-jam: desenrola e combina laços para evitar desvios
 - ▶ -fsplit-paths: divide caminhos que levam a laços de retorno, melhorando a remoção de código morto e eliminação de subexpressão comum

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 3.599 s +- 0.152 s [User: 3.595 s, System: 0.002 s]
Range (min ... max): 3.512 s ... 3.976 s 10 runs
```

Diretivas de compilação

- ▶ Otimização de desempenho (-Ofast)
 - ▶ Realiza todas as otimizações de -O3 e aplica otimizações -ffast-math que não são válidas dentro dos padrões de conformidade
 - ▶ -funsafe-math-optimizations: assume que os argumentos e os resultados são válidos, podendo gerar violações nos padrões IEEE e ANSI
 - ▶ -ffinite-math-only: assume que não serão usados valores indefinidos ou infinitos, sendo possível a geração de resultados incorretos
 - ▶ -fno-rounding-math: permite que as expressões de ponto flutuante sejam avaliadas em tempo de compilação, sem mudanças dinâmicas no modo de arredondamento

```
$ gcc -Wall -Ofast exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
```

Diretivas de compilação

- ▶ Otimização de desempenho (-Ofast)
 - ▶ Realiza todas as otimizações de -O3 e aplica otimizações -ffast-math que não são válidas dentro dos padrões de conformidade
 - ▶ -funsafe-math-optimizations: assume que os argumentos e os resultados são válidos, podendo gerar violações nos padrões IEEE e ANSI
 - ▶ -ffinite-math-only: assume que não serão usados valores indefinidos ou infinitos, sendo possível a geração de resultados incorretos
 - ▶ -fno-rounding-math: permite que as expressões de ponto flutuante sejam avaliadas em tempo de compilação, sem mudanças dinâmicas no modo de arredondamento

```
$ gcc -Wall -Ofast exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 3.748 s +- 0.179 s [User: 3.595 s, System: 0.002 s]
Range (min ... max): 3.528 s ... 4.075 s 10 runs
```

Diretivas de compilação

- ▶ Otimização de espaço (-Os)
 - ▶ Reduz o tamanho do código gerado, aplicando as otimizações -O2 que não aumentam o tamanho
 - ▶ Habilita -finline-functions para ajustar o tamanho do código e não para melhorar o desempenho

```
$ gcc -Wall -Os exemplo.c -o exemplo.elf  
$ hyperfine --warmup 3 './exemplo.elf'
```

Diretivas de compilação

- ▶ Otimização de espaço (-Os)
 - ▶ Reduz o tamanho do código gerado, aplicando as otimizações -O2 que não aumentam o tamanho
 - ▶ Habilita -finline-functions para ajustar o tamanho do código e não para melhorar o desempenho

```
$ gcc -Wall -Os exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
  Time (mean +- d): 8.557 s +- 0.447 s [User: 3.595 s, System: 0.002 s]
  Range (min ... max): 8.167 s ... 9.384 s 10 runs
```

Diretivas de compilação

- ▶ Otimização para depuração (-Og)
 - ▶ Melhora a experiência de depuração, aplicando as otimizações -O1 que não afetam a depuração
 - ▶ Ideal para a etapa de desenvolvimento

```
$ gcc -Wall -Og exemplo.c -o exemplo.elf  
$ hyperfine --warmup 3 './exemplo.elf'
```


Diretivas de compilação

- ▶ Otimização para depuração (-Og)
 - ▶ Melhora a experiência de depuração, aplicando as otimizações -O1 que não afetam a depuração
 - ▶ Ideal para a etapa de desenvolvimento

```
$ gcc -Wall -Og exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 2.384 s +- 0.074 s [User: 3.595 s, System: 0.002 s]
Range (min ... max): 2.344 s ... 2.558 s 10 runs
```

Diretivas de compilação

- Comparativo de desempenho relativo das diretivas

Diretiva	Tempo (s)	Desempenho
-O0	$9,300 \pm 0,533$	1,00x
-O1	$8,253 \pm 0,078$	1,13x
-O2	$8,179 \pm 0,023$	1,14x
-O3	$3,599 \pm 0,152$	2,58x
-Ofast	$3,748 \pm 0,179$	2,48x
-Os	$8,557 \pm 0,447$	1,09x
-Og	$2,384 \pm 0,074$	3,90x

Eficiência algorítmica

- Ordenação por força bruta é $\Theta(n^2)$

```
1  #include <stdio.h>
...
8  void ordenar(int32_t* V, uint32_t n) {
9      for(uint32_t i = 0; i < n - 1; i++) {
10         uint32_t min = i;
11         for(uint32_t j = i + 1; j < n; j++)
12             if(V[j] < V[min]) min = j;
13         if(i != min) trocar(&V[i], &V[min]);
14     }
15 }
...
```

Eficiência algorítmica

- Ordenação por força bruta é $\Theta(n^2)$

```
1  #include <stdio.h>
...
8  void ordenar(int32_t* V, uint32_t n) {
9      for(uint32_t i = 0; i < n - 1; i++) {
10         uint32_t min = i;
11         for(uint32_t j = i + 1; j < n; j++)
12             if(V[j] < V[min]) min = j;
13         if(i != min) trocar(&V[i], &V[min]);
14     }
15 }
...
```

É possível otimizar este algoritmo?

Eficiência algorítmica

► Ordenação com Heapsort é $\Theta(n \log_2 n)$

```
1  #include <stdio.h>
...
8  void heapify(int32_t* V, uint32_t i, uint32_t n) {
9      uint32_t P = i, E = 2 * i + 1, D = 2 * i + 2;
10     for(uint32_t j = E; j < n && j <= D; j++)
11         if(V[j] > V[P]) P = j;
12     if(P != i) {
13         trocar(&V[P], &V[i]);
14         heapify(V, P, n);
15     }
16 }
17 void construir_heap(int32_t* V, uint32_t n) {
18     for(int32_t i = n / 2; i >= 0; i--)
19         heapify(V, i, n);
20 }
...
...
```

Eficiência algorítmica

- ▶ Ordenação com Heapsort é $\Theta(n \log_2 n)$

```
1  #include <stdio.h>
...
21 void ordenar(int32_t* V, uint32_t n) {
22     construir_heap(V, n);
23     for(uint32_t i = n - 1; i > 0; i--) {
24         trocar(&V[0], &V[i]);
25         heapify(V, 0, i);
26     }
27 }
28 int main() {
29     const uint32_t n = 100000;
30     int32_t* V = (int32_t*)(malloc(n *
31         sizeof(int32_t)));
32     for(uint32_t i = 0; i < n; i++)
33         V[i] = rand() * rand();
34     ordenar(V, n);
35     printf("min_=%i, max_=%i\n", V[0], V[n - 1]);
36     return 0;
}
```

Eficiência algorítmica

- Comparativo de desempenho da ordenação por força bruta versus Heapsort

Diretiva	Força bruta (s)	Heapsort (ms)	Melhoria
-O0	$9,300 \pm 0,533$	$26,5 \pm 0,2$	351x
-O1	$8,253 \pm 0,078$	$9,5 \pm 0,1$	869x
-O2	$8,179 \pm 0,023$	$10,4 \pm 0,1$	786x
-O3	$3,599 \pm 0,152$	$11,5 \pm 0,1$	313x
-Ofast	$3,748 \pm 0,179$	$11,5 \pm 0,1$	326x
-Os	$8,557 \pm 0,447$	$10,0 \pm 0,1$	856x
-Og	$2,384 \pm 0,074$	$16,5 \pm 0,2$	145x

Eficiência algorítmica

- Comparativo de desempenho da ordenação por força bruta versus Heapsort

Diretiva	Força bruta (s)	Heapsort (ms)	Melhoria
-O0	$9,300 \pm 0,533$	$26,5 \pm 0,2$	351x
-O1	$8,253 \pm 0,078$	$9,5 \pm 0,1$	869x
-O2	$8,179 \pm 0,023$	$10,4 \pm 0,1$	786x
-O3	$3,599 \pm 0,152$	$11,5 \pm 0,1$	313x
-Ofast	$3,748 \pm 0,179$	$11,5 \pm 0,1$	326x
-Os	$8,557 \pm 0,447$	$10,0 \pm 0,1$	856x
-Og	$2,384 \pm 0,074$	$16,5 \pm 0,2$	145x

É possível otimizar ainda mais?

Análise de desempenho

- ▶ É preciso aumentar o tempo de execução com uma entrada maior para permitir a coleta dos dados

```
1  #include <stdio.h>
...
28 int main() {
29     const uint32_t n = 100000000;
30     int32_t* V = (int32_t*)(malloc(n *
        sizeof(int32_t)));
31     for(uint32_t i = 0; i < n; i++)
32         V[i] = rand() * rand();
33     ordenar(V, n);
34     printf("min_ = %i, max_ = %i\n", V[0], V[n - 1]);
35     return 0;
36 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
```

Análise de desempenho

- ▶ É preciso aumentar o tempo de execução com uma entrada maior para permitir a coleta dos dados

```
1  #include <stdio.h>
...
28 int main() {
29     const uint32_t n = 100000000;
30     int32_t* V = (int32_t*)(malloc(n *
        sizeof(int32_t)));
31     for(uint32_t i = 0; i < n; i++)
32         V[i] = rand() * rand();
33     ordenar(V, n);
34     printf("min_ = %i, max_ = %i\n", V[0], V[n - 1]);
35     return 0;
36 }
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 2.755 s +- 0.013 s [User: 2.740 s, System: 0.012 s]
Range (min ... max): 2.741 s ... 2.783 s 10 runs
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf  
$ ./exemplo.elf
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf  
$ ./exemplo.elf  
min = -2147483600, max = 2147483400
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gprof -l exemplo.elf
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative  self    ...   name
time   seconds  seconds  ...
62.58      1.60      1.60    ... heapify (exemplo.c:11 @ 14c3)
15.25      1.98      0.39    ... heapify (exemplo.c:11 @ 1495)
 7.92      2.19      0.20    ... trocar (exemplo.c:6 @ 14e3)
 3.56      2.28      0.09    ... heapify (exemplo.c:10 @ 14ba)
 1.98      2.33      0.05    ... heapify (exemplo.c:11 @ 1413)
 1.98      2.38      0.05    ... heapify (exemplo.c:12 @ 14de)
 1.58      2.42      0.04    ... heapify (exemplo.c:9 @ 1480)
 1.19      2.45      0.03    ... heapify (exemplo.c:11 @ 13ed)
 0.79      2.47      0.02    ... heapify (exemplo.c:10 @ 148c)
 0.79      2.49      0.02    ... heapify (exemplo.c:11 @ 14b6)
 0.79      2.51      0.02    ... ordenar (exemplo.c:23 @ 14f0)
 0.40      2.52      0.01    ... heapify (exemplo.c:10 @ 14d3)
 0.40      2.53      0.01    ... ordenar (exemplo.c:23 @ 1456)
 0.40      2.54      0.01    ... trocar (exemplo.c:5 @ 1470)
 0.20      2.55      0.01    ... heapify (exemplo.c:8 @ 147f)
 0.20      2.55      0.01    ... heapify (exemplo.c:11 @ 14d6)
...
```

Análise de desempenho

► Procedimentos *trocar* e *heapify*

```
1  #include <stdio.h>
...
4  void trocar(int32_t* i, int32_t* j) {
5      int32_t k = *i;
6      *i = *j; *j = k;
7  }
8  void heapify(int32_t* V, uint32_t i, uint32_t n) {
9      uint32_t P = i, E = 2 * i + 1, D = 2 * i + 2;
10     for(uint32_t j = E; j < n && j <= D; j++)
11         if(V[j] > V[P]) P = j;
12     if(P != i) {
13         trocar(&V[P], &V[i]);
14         heapify(V, P, n);
15     }
16 }
...
...
```

Análise de desempenho

► Procedimentos *trocar* e *heapify*

```
1  #include <stdio.h>
...
4  void trocar(int32_t* i, int32_t* j) {
5      int32_t k = *i;
6      *i = *j; *j = k;
7  }
8  void heapify(int32_t* V, uint32_t i, uint32_t n) {
9      uint32_t P = i, E = 2 * i + 1, D = 2 * i + 2;
10     for(uint32_t j = E; j < n && j <= D; j++)
11         if(V[j] > V[P]) P = j;
12     if(P != i) {
13         trocar(&V[P], &V[i]);
14         heapify(V, P, n);
15     }
16 }
...
...
```

As linhas 6, 9, 10, 11 e 12 concentram
pelo menos 97% do tempo de execução

Análise de desempenho

- ▶ Otimizando o procedimento *heapify*
 - ▶ Substituindo o controle iterativo por condicional

```
1  #include <stdio.h>
...
8  void heapify(int32_t* V, uint32_t i, uint32_t n) {
9      uint32_t P = i, E = 2 * i + 1, D = 2 * i + 2;
10     if(E < n && V[E] > V[P]) P = E;
11     if(D < n && V[D] > V[P]) P = D;
12     if(P != i) {
13         trocar(&V[P], &V[i]);
14         heapify(V, P, n);
15     }
16 }
...
...
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
```

Análise de desempenho

- ▶ Otimizando o procedimento *heapify*
 - ▶ Substituindo o controle iterativo por condicional

```
1  #include <stdio.h>
...
8  void heapify(int32_t* V, uint32_t i, uint32_t n) {
9      uint32_t P = i, E = 2 * i + 1, D = 2 * i + 2;
10     if(E < n && V[E] > V[P]) P = E;
11     if(D < n && V[D] > V[P]) P = D;
12     if(P != i) {
13         trocar(&V[P], &V[i]);
14         heapify(V, P, n);
15     }
16 }
...
...
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 2.380 s +- 0.013 s [User: 2.371 s, System: 0.007 s]
Range (min ... max): 2.368 s ... 2.413 s 10 runs
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf  
$ ./exemplo.elf
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf  
$ ./exemplo.elf  
min = -2147483600, max = 2147483400
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gprof -l exemplo.elf
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative   self           ...      name
time  seconds    seconds           ...
53.24    1.17    1.17           ... heapify (exemplo.c:10 @ 1298)
24.77    1.71    0.54           ... heapify (exemplo.c:11 @ 12d2)
 7.41    1.87    0.16           ... trocar (exemplo.c:6 @ 12ba)
 5.56    1.99    0.12           ... heapify (exemplo.c:11 @ 12aa)
 4.17    2.08    0.09           ... heapify (exemplo.c:13 @ 12b4)
 1.39    2.11    0.03           ... heapify (exemplo.c:9 @ 12c3)
 1.16    2.14    0.03           ... heapify (exemplo.c:10 @ 12cd)
 0.93    2.16    0.02           ... heapify (exemplo.c:8 @ 1280)
 0.46    2.17    0.01           ... heapify (exemplo.c:12 @ 12af)
 0.46    2.18    0.01           ... trocar (exemplo.c:7 @ 1273)
 0.46    2.19    0.01           ... trocar (exemplo.c:6 @ 1390)
 0.46    2.20    0.01           ... trocar (exemplo.c:5 @ 1394)
 0.23    2.21    0.01           ... ordenar (exemplo.c:25 @ 13a5)
 0.23    2.21    0.01           ... trocar (exemplo.c:6 @ 13a1)
...
```

Análise de desempenho

- ▶ Otimizando o procedimento *trocar*
 - ▶ Sintaxe padrão AT&T/Unix

```
1 #include <stdio.h>
...
4 #define trocar(i, j)\
5     __asm__(\
6         "mov_(%0),_%%eax;"\
7         "mov_(%1),_%%ecx;"\
8         "mov_%%eax,_(%1);" \
9         "mov_%%ecx,_(%0);" \
10        : \
11        : "r"(i), "r"(j)\
12        : "eax", "ecx", "memory" \
13    )
...
...
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
```

Análise de desempenho

- ▶ Otimizando o procedimento *trocar*
- ▶ Sintaxe padrão AT&T/Unix

```
1 #include <stdio.h>
...
4 #define trocar(i, j)\
5     __asm__(\
6         "mov_(%0),_%%eax;"\
7         "mov_(%1),_%%ecx;"\
8         "mov_%%eax,_(%1);" \
9         "mov_%%ecx,_(%0);" \
10        : \
11        : "r"(i), "r"(j)\
12        : "eax", "ecx", "memory" \
13    )
...
...
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 2.206 s +- 0.012 s [User: 2.192 s, System: 0.012 s]
Range (min ... max): 2.189 s ... 2.223 s 10 runs
```


Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf  
$ ./exemplo.elf
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gcc -Wall -O3 -g -pg exemplo.c -o exemplo.elf  
$ ./exemplo.elf  
min = -2147483600, max = 2147483400
```

Análise de desempenho

► Perfil de execução (gprof)

```
$ gprof -l exemplo.elf
Flat profile:

Each sample counts as 0.01 seconds.
 %   cumulative   self           ...      name
time  seconds    seconds           ...
32.93      0.69      0.69           ... heapify (exemplo.c:17 @ 1291)
29.81      1.32      0.63           ... heapify (exemplo.c:16 @ 1276)
12.98      1.60      0.27           ... heapify (exemplo.c:17 @ 12d1)
10.58      1.82      0.22           ... heapify (exemplo.c:19 @ 129c)
 4.57      1.92      0.10           ... heapify (exemplo.c:18 @ 1298)
 4.57      2.01      0.10           ... heapify (exemplo.c:17 @ 12ba)
 1.68      2.05      0.04           ... heapify (exemplo.c:16 @ 12b5)
 1.44      2.08      0.03           ... heapify (exemplo.c:15 @ 12ac)
 0.96      2.10      0.02           ... heapify (exemplo.c:22 @ 12cf)
 0.48      2.11      0.01           ... heapify (exemplo.c:15 @ 126d)
 0.48      2.12      0.01           ... ordenar (exemplo.c:29 @ 1387)
 0.00      2.12      0.00           ... heapify (exemplo.c:14 @ 1260)
...
$
```

Análise de desempenho

- ▶ Otimizando o procedimento *heapify*
 - ▶ Implementação iterativa

```
1  #include <stdio.h>
...
14 void heapify(int32_t* V, uint32_t i, uint32_t n) {
15     while(i < n) {
16         uint32_t P = i, E = (P << 1) + 1, D = E + 1;
17         if(E < n && V[E] > V[i]) i = E;
18         if(D < n && V[D] > V[i]) i = D;
19         if(P == i) break;
20         trocar(&V[P], &V[i]);
21     }
22 }
...
...
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
```

Análise de desempenho

- ▶ Otimizando o procedimento *heapify*
 - ▶ Implementação iterativa

```
1  #include <stdio.h>
...
14 void heapify(int32_t* V, uint32_t i, uint32_t n) {
15     while(i < n) {
16         uint32_t P = i, E = (P << 1) + 1, D = E + 1;
17         if(E < n && V[E] > V[i]) i = E;
18         if(D < n && V[D] > V[i]) i = D;
19         if(P == i) break;
20         trocar(&V[P], &V[i]);
21     }
22 }
...
...
```

```
$ gcc -Wall -O3 exemplo.c -o exemplo.elf
$ hyperfine --warmup 3 './exemplo.elf'
Benchmark #1: ./exemplo.elf
Time (mean +- d): 2.247 s +- 0.007 s [User: 2.237 s, System: 0.008 s]
Range (min ... max): 2.238 s ... 2.258 s 10 runs
```

Análise de desempenho

- ▶ Pontos chave da otimização de código
 - ▶ Esforço $\times$ Legibilidade do código $\times$ Melhoria

Análise de desempenho

- ▶ Pontos chave da otimização de código
 - ▶ Esforço $\times$ Legibilidade do código $\times$ Melhoria
 - ▶ Evite inserir erros com testes de regressão

Análise de desempenho

- ▶ Pontos chave da otimização de código
 - ▶ Esforço $\times$ Legibilidade do código $\times$ Melhoria
 - ▶ Evite inserir erros com testes de regressão
 - ▶ Sempre verifique o desempenho do código (*profiling*)

Exercício

- ▶ Reproduza os passos de otimização vistos nesta aula para o algoritmo de ordenação Mergesort
 - ▶ Compare a implementação iterativa versus recursiva
 - ▶ Analise a implementação em código de montagem para trechos com grande impacto no tempo de execução, buscando inferir um código mais eficiente
 - ▶ Explique o que seria o ponto ótimo de desempenho